

Excavator3000 DQN Training Upgrade Log

Date: 2026-08-17

Branch: `feature/dqn-curriculum-v2`

Outcome

The training foundation now uses a smaller three-action driving curriculum, repeated actions, ordered track progress, explicit stuck termination, Double DQN targets, scaled rewards, success-aware checkpoints, and richer dashboard data. A physical Webots diagnostic verified that both turn commands rotate the simulated robot in opposite directions.

The current long training run uses Webots fast mode with rendering disabled. The local dashboard remains independent on port `8080`, so Webots can train without a graphical window while training data is still visible in a browser or Cloudflare Tunnel.

Why this upgrade was needed

The previous seven-action run learned weak behavior. The old checkpoint frequently selected stop or reverse actions and did not demonstrate reliable forward turns. Its training data also could not distinguish meaningful track progress from simply surviving. A reward score could therefore change without showing whether the robot learned the route.

This upgrade makes the first curriculum easier to learn and makes the evidence easier to interpret. The robot must first learn forward driving and left/right steering. Reverse, stop, and recovery actions can be added in a later curriculum after basic navigation is stable.

The seven implemented improvements

1. Verify physical steering before training

`rl_controller.py` now provides a `diagnostic` mode that runs the real Webots motors and measures angular velocity. The diagnostic resets the robot before each command and checks that left and right turns both exceed a minimum rate and have opposite signs.

Measured result:

Command	Mean turn rate	Result
Turn left	+0.565 rad/s	Pass
Turn right	-0.555 rad/s	Pass

This confirms the configured wheel ratios create physical turns rather than only changing an action label.

2. Start with a three-action curriculum

The policy action space is now:

Policy action	Robot action	Meaning
0	0	Forward
1	1	Turn left while moving
2	2	Turn right while moving

`environment.py` maps these policy actions to the full robot action table. This keeps the robot controller reusable while the first DQN has only three outputs and 5,059 trainable network parameters.

3. Repeat each decision for four physics steps

Each DQN decision is held for four Webots steps. Rewards are accumulated across those steps, but collision, goal, stuck, and timeout conditions are checked every physics step. A terminal event therefore stops the repeat immediately instead of delaying safety checks.

This gives steering actions enough time to affect motion and reduces the number of nearly identical replay transitions.

4. Add ordered route progress and checkpoint rewards

`progress_tracker.py` tracks ten ordered waypoints from the starting lane through the turns toward the goal. It provides:

- Clipped distance-change shaping toward the active waypoint.
- A one-time reward when the next ordered checkpoint is reached.
- Checkpoint count and route completion fraction.
- Best-distance tracking for stuck detection.

The goal remains the authoritative success condition. Waypoints teach direction and do not replace the finish line.

5. Add explicit stuck termination

`stuck` is now a first-class termination reason alongside `collision`, `goal_reached`, `timeout`, and `simulation_stopped`. An episode terminates as stuck when it makes no new best progress for the configured number of physics steps.

The stuck penalty is separate from collision and timeout. This prevents long unproductive episodes while preserving clear diagnostic data.

6. Use Double DQN and scaled learning targets

The online network now chooses the best next action while the target network evaluates that action. This is the Double DQN target and reduces the overestimation produced by taking the maximum directly from the target network.

Rewards are multiplied by `0.01` only when constructing learning targets. CSV and dashboard rewards stay in their original human-readable units. Collision `-100`, stuck `-60`, timeout `-75`, and goal rewards therefore remain easy to interpret while neural-network targets stay numerically small.

7. Improve checkpoints, logging, and the dashboard

Model saving is now success-aware:

- `dqn_best.pt` is written only by a successful goal-reaching episode.

- `dqn_candidate.pt` stores the strongest unsuccessful episode for diagnosis.
- `dqn_latest.pt` stores periodic resumable training state.

The CSV and dashboard now include physics steps, decisions, mean reward per step, closest/final goal distance, track progress, checkpoints reached, and forward/left/right action percentages. The dashboard reads the configured run directory and displays the latest action mix and route progress.

Automated verification

The complete automated suite passes:

```
Ran 47 tests in 0.957s
OK
```

Coverage includes action mapping and repeat behavior, terminal interruption during a repeat, ordered progress, stuck detection, Double DQN target selection, reward scaling, success-aware model saving, logger fields, dashboard configuration, replay buffer behavior, observations, and reward calculations.

Headless 200-episode calibration

The calibration deliberately began near 100% exploration, so its collision rate is not an evaluation of the final policy. Its purpose was to prove that the environment, rewards, progress tracker, replay learning, and logging work together.

Metric	Result
Episodes	200
Total DQN decisions	34,018
Mean decisions per episode	170.1
Terminations	177 collision, 23 stuck
Maximum ordered checkpoints	3 of 10
First 50 mean reward	-92.966
Last 50 mean reward	-92.000
First 50 mean physics steps	673.5
Last 50 mean physics steps	741.6
First 50 mean progress	0.074
Last 50 mean progress	0.078
Final epsilon	0.9892

Random exploration reached three ordered checkpoints, demonstrating that the route and checkpoint ordering are physically reachable. Replay learning started after the 2,000-transition warmup, and loss values remained finite.

Evidence-based tuning decision

The initial epsilon decay was `3,000,000` DQN decisions. Calibration measured about 170 decisions per episode, projecting only about 1.70 million decisions across 10,000 similar episodes. Epsilon would still be approximately `0.46` at the end, leaving the policy too exploratory.

The decay was changed to `1,000,000` decisions. At the measured decision rate, epsilon should reach its `0.05` floor around episode 5,900. This leaves roughly 4,100 episodes for mostly greedy improvement and observation. No reward, learning-rate, or terminal-penalty change was justified by the short high-exploration calibration.

Current training configuration

Setting	Value
Webots mode	Fast, no rendering
Episodes	10,000
Maximum physics steps	3,000
Actions	Forward, left, right
Action repeat	4
Replay capacity	50,000
Replay warmup	2,000 decisions
Batch size	64
Discount factor	0.99
Learning rate	0.0003
Double DQN	Enabled
Reward scale	0.01
Epsilon	1.00 to 0.05 over 1,000,000 decisions
Target sync	Every 1,000 learning updates
Checkpoint radius	0.55 m
Stuck window	400 physics steps without 0.03 m new progress

Running headlessly from PowerShell

```
$env:RL_PROGRAM_MODE = "train"
$env:RL_TRAINING_EPISODES = "10000"
& "C:\Program Files\Webots\msys64\mingw64\bin\webots.exe" `
  --batch --mode=fast --no-rendering --stdout --stderr `
  "C:\Users\titan\OneDrive\Documents\Bot_sim\worlds\car_env_test_v2.wbt"
```

`--mode=fast` removes real-time pacing. `--no-rendering` disables the 3D view, and `--batch` prevents blocking pop-up windows. The dashboard is a separate process and can remain running at `http://127.0.0.1:8080` or through a Cloudflare Tunnel whose origin is `http://localhost:8080`.

Monitoring rules for the long run

Evaluate rolling windows, not individual episodes:

- Check checkpoint progress and successful goals before judging reward alone.
- Compare the most recent 100 episodes with the previous 100.
- Interpret collision rate together with epsilon; high collision at high epsilon is expected exploration.
- Verify that action percentages do not collapse permanently to one action.
- Treat falling loss as supporting evidence, not the main measure of driving quality.
- Change one training parameter at a time and preserve the previous run/checkpoint.

Live run health check

At episode 502, the tuned run had epsilon `0.9215`, finite loss, and an overall best of three ordered checkpoints. The latest 100 episodes averaged `-94.56` reward, 644 physics steps, and `0.072` route progress, with 90 collisions and 10 stuck terminations. The policy was still strongly exploratory, so this window did not justify another parameter change. Webots, the Python training controller, and the port 8080 dashboard were all confirmed active.

Sim-to-real boundary

The DQN learns a discrete decision: forward, turn left, or turn right. The ESP32 hardware layer should map that decision to the real TT-motor PWM values. Speed remains part of the observation, so later training can expose the policy to multiple simulated speed scales. The hardware team should still tune acceleration, minimum usable PWM, left/right mismatch, braking, and safety limits on the real 68 mm-wheel vehicle.

The next curriculum should add randomized starts, randomized headings, multiple tracks, sensor noise, and speed variation only after this basic route produces consistent goal-reaching checkpoints.